



UNIVERSIDADE ESTADUAL PAULISTA
"JÚLIO DE MESQUITA FILHO"

INSTITUTO DE CIÊNCIA E TECNOLOGIA DE SOROCABA

ENGENHARIA DE CONTROLE E AUTOMAÇÃO

ESTRUTURA DE DADOS

Lucas Ribeiro Moschiar

RA: 251270114 ^a

SISTEMA DE GERENCIAMENTO E ANÁLISE DE DADOS PARA OFICINA DE MOTOS

Sorocaba

2026

Lucas Ribeiro Moschiar

RA: 251270114^a

SISTEMA DE GERENCIAMENTO E ANÁLISE DE DADOS PARA OFICINA DE MOTOS

Trabalho apresentado à disciplina de Estrutura de Dados, do curso de Engenharia de Controle e Automação, como parte dos requisitos avaliativos do projeto de Estrutura de Dados.

Professor: Leopoldo André Dutra Lusquino Filho

Sorocaba

2026

Resumo

O presente trabalho tem como objetivo um sistema em linguagem C++ para um gerenciamento e análise de um catálogo de peças de motos, aplicando diferentes estruturas de dados. O sistema utiliza um dataset estruturado em arquivo TXT, contendo informações como código da peça, descrição, nome, categoria, fabricante, modelo da moto, preço, quantidade em estoque e quantidade mínima. Os dados foram implementados e complementaram operações de busca, inserção, remoção, filtragem por categoria, listagem de estoque crítico, cálculo de estatísticas e geração de benchmarks. Foram implementadas estruturas tradicionais como Lista Encadeada, Tabela Hash e Árvore AVL, além de estruturas auxiliares Trie e Bloom Filter. Foi desenvolvida uma Hash otimizada com redimensionamento automático, para fim de comparar seu comportamento com uma Tabela Hash simples. Cada estrutura foi aplicada de acordo com suas finalidades, evitando uma comparação inadequada referente ao objetivo da estrutura. Os resultados foram analisados por meio de benchmarks de busca, inserção e remoção, além da presença de condições restritivas de memória, processamento, latência, dados e algorítmica. De modo geral, o sistema analisa e propõe a escolha de diferentes estruturas de dados e sua influência no desempenho diante da comparação e limitação de cada abordagem estrutural.

Palavras-chave: Estruturas de dados; Lista Encadeada; Tabela Hash; Árvore AVL; Trie; Bloom Filter.

Sumário

1	Introdução	4
2	Referencial Teórico	4
2.1	Lista Encadeada	5
2.2	Tabela Hash	5
2.3	Hash Otimizada	5
2.4	Árvore AVL	5
2.5	Trie	6
2.6	Bloom Filter	6
2.7	Benchmarks	6
2.8	Teste de Restrições	7
3	Metodologia	7
3.1	Descrição do sistema	7
3.2	Preparação do dataset	8
3.3	Tratamento dos dados	8
3.4	Operação do sistema	9
4	Desenvolvimento do Sistema	9
4.1	Atuação dos registros	9
4.2	Inserção de dados	10
4.3	Busca de dados	10
4.4	Remoção de dados	10
4.5	Visualização dos dados	11
5	Benchmarks e Resultados	11
5.1	Busca por código	11
5.2	Tabela Hash e Hash Otimizada	12
5.3	Benchmark Lista e Trie	12
5.4	Benchmark Bloom Filter e Tabela Hash	13
5.5	Resultados de inserção e remoção	13
5.6	Memória Aproximada	14
6	Testes com Restrições	15
6.1	Restrição de memória	15
6.2	Restrição de processamento	15
6.3	Restrição de latência	16
6.4	Restrição de dados	16
6.5	Restrição algorítmica e estrutural	17

6.6 Escalabilidade	17
7 Conclusão	17
Referências	19

1 Introdução

Pelos inúmeros dados fornecidos e gerados por sistemas computacionais, o uso de estrutura de dados é fundamental para o armazenamento, busca, organização e análise estatística. Diante das aplicações, os dados precisam ser manipulados de forma eficiente, destes sistemas de estoque, catálogos comerciais, plataformas de vendas e sistemas industriais, o que permite a inserção, consulta, remoção e captação útil de dados para tomada de decisões.

Sendo assim, o trabalho tem como objetivo desenvolver um sistema em linguagem C++ que gerencia dados de um catálogo de peças de motos. O sistema utiliza um dataset estruturado em um arquivo TXT, sobre análise de informações do código de peça, descrição, nome, categoria, fabricante, modelo, preço, quantidade e quantidade mínima. A análise desses dados representa uma demonstração de estrutura de dados, demonstrando as diferentes aplicações de cadastro, busca, filtros e análises estatísticas.

Para a proposta do trabalho, foram implementadas diferentes estruturas de dados, que realizaram manipulação diante de comportamentos de busca, inserção e remoção. Foram utilizadas Lista Encadeada, Tabela Hash e Árvore AVL, além de estruturas auxiliares como Trie e Bloom Filter, que redimensionaram a ideia de busca, quanto por prefixos de nomes, até a verificação probabilística do código. Também foi implementada uma versão otimizada da Tabela Hash, redimensionando a estrutura de maneira automática quando ultrapassado determinado nível de ocupação, permitindo comparar a estrutura original com a versão modificada.

O sistema realça outras ações, filtragem por categoria, estatística do estoque crítico, listagem de preços, geração de benchmarks e testes em condições restritivas. Esses testes buscam avaliar aspectos operacionais de tempo de busca, tempo de inserção, remoção, colisão nas estruturas, memória utilizada, escalabilidade e comportamentos das estruturas limitadas, reforçando o processamento, a latência e a qualidade dos dados.

Nesse sentido, o presente trabalho tem o objetivo de analisar de forma prática e conceitual diferentes estruturas de dados, representando a análise de comportamento diante do mesmo conjunto de informações. A proposta permite analisar a teoria estudada com a aplicação definitiva, observando as vantagens e limitações envolvidas no comportamento de cada estrutura.

2 Referencial Teórico

Estrutura de dados realça a forma de organização e manipulação de informações, as diferentes estruturas utilizadas influenciam diretamente no desempenho do sistema, diante das inúmeras inserções, remoções, busca, filtragem e análise de dados. Com a análise do sistema de catálogo de peças, a consulta de inúmeros registros reforça o uso de diferentes

estruturas para comparar o comportamento operacional semelhante, viabilizando uma complexidade operacional e possibilidade do desempenho teórico ao prático das estruturas.

2.1 Lista Encadeada

A lista encadeada é uma estrutura que apresenta nós, cuja ideia é armazenar uma informação em cada nó e um ponteiro para o próximo elemento, viabilizando uma inserção, remoção e até a busca dos dados. Diante do sistema, a lista foi aplicada na forma de comparação diante do código de cada peça, onde a busca exige percorrer os nós até encontrar o nó correspondente, viabilizando uma complexidade $O(n)$. Isso significa um tempo maior mediante ao número de peças cadastradas.

2.2 Tabela Hash

A tabela hash é uma estrutura que utiliza uma função hash transformando uma chave em uma posição da tabela, calculando uma posição a partir de um dado referência, permitindo o acesso e registro.

No sistema, a chave utilizada é o código da peça, com inúmeros posições calculadas é notório a ideia de colisão, o que acontece quando duas ou mais chaves diferentes são direcionadas para a mesma posição. A ideia de tratar as colisões é realçado de maneira comum por encadeamento separado, posicionando uma lista ligada de elementos em cada posição da tabela, onde a busca dessa tabela possui uma complexidade media $O(1)$ que mantém o acesso pela posição calculada.

2.3 Hash Otimizada

Essa versão consiste em controlar o fator de carga da tabela, redimensionando o tamanho por meio da quantidade de dados. Idealizando o processo de rehash, onde os elementos são inseridos em uma tabela maior, que recalcula as posições e evita a tendência de colisões.

Todavia, o custo de operação da estrutura aumenta pela reorganização dos elementos, mesmo assim, a adaptação é vantajosa em um sistema de crescimento de dados. No trabalho, a Hash otimizada foi utilizada para meios comparativos, idealizando uma diferença da estrutura com a versão base, permitindo compreender a adaptação ao crescimento do dataset entre uma estrutura de tamanho fixo.

2.4 Árvore AVL

A árvore AVL é uma árvore binária de busca balanceada, onde cada nó possui uma chave, no qual os elementos ficam de maneira organizada, sendo os dados menores a esquerda e os maiores a direita, permitindo realizar buscas mais previsíveis e eficientes.

No sistema desenvolvido, a árvore AVL foi moldada pela ideia tradicional de busca pelo código da peça, oferecendo um desempenho de complexidade $O(\log n)$, sobre uma organização e rotação AVL que verifica o fator de controle e balanceamento dos nós, mediante a inserção ou remoção dos dados.

2.5 Trie

A Trie é uma estrutura de dados, especializada no armazenamento e na busca de strings, organizando cada dado pelo caractere, onde cada caminho da raiz determina um prefixo da palavra armazenada.

No projeto, a estrutura foi mantida como auxiliar, o que complementa o sistema por consultas textuais, representando uma adequação pelos nomes de peças, sobre uma pesquisa eficiente dos prefixos. A complexidade de busca da Trie é mantida pelo tamanho da palavra e não diretamente pela quantidade total de dados.

2.6 Bloom Filter

O Bloom filter é uma estrutura probabilística, que verifica se um elemento pertence a um conjunto, é utilizado um vetor de bits e múltiplas funções hash, quando um elemento é inserido, algumas posições do vetor são marcadas, quando consultadas essas posições são verificadas e se corresponde a marcação. O Bloom filter pode apresentar a ideia do falso positivo, é trabalhado de maneira probabilística, assim não indica um falso negativo.

Diante do sistema, essa estrutura é mantida como auxiliar, o que informa uma verificação rápida se um código da peça provavelmente existe, entretanto, a confirmação real é feita pela tabela hash, tal combinação permite a demonstração de consumo de memória e é adequada para teste eficaz da existência de dados.

2.7 Benchmarks

Um benchmark é um processo de comparar o desempenho sobre condições semelhantes, de maneira geral, é uma comparação entre estruturas. No trabalho, os benchmarks são utilizados para avaliar tempo de busca, inserção, remoção, uso de memória, escalabilidade, além do desempenho ideal ao prático, essas comparações devem ser feitas com cuidado.

A principal comparação é mantida entre a lista encadeada, a tabela hash, a hash otimizada e a árvore AVL, no qual são estruturas semelhantes de busca pelo código das peças. Já as estruturas auxiliares são analisadas de maneira diferente, por causa da finalidade dentro do sistema, idealizando as buscas por prefixo textual e pela probabilística de existência de dados.

2.8 Teste de Restrições

Os testes de restrições favorecem a ideia de avaliar o comportamento das estruturas em diferentes condições, observando o comportamento do sistema por diversos meios e limitações. No trabalho, a organização de restrições foi mantida em uma linha de categorias sobre aplicações na memória, no processamento, na latência, na qualidade de dados e na eficiência.

3 Metodologia

A metodologia aplicada teve como objetivo introduzir diferentes estruturas de dados sobre uma ação prática, permitindo a realização de inserção, remoção, busca, filtragem, análises estatísticas, meios comparativos e condições restritivas, onde foi utilizado um conjunto de bibliotecas padrão da linguagem C++ (Standard Template Library - STL). A ideia foi analisar um catálogo de peças de motos, o que representa aplicação real no conceito de armazenar inúmeros registros, além da disponibilidade de estoque por consultas, análise de categorias e identificar o potencial de peças em estoque.

3.1 Descrição do sistema

O sistema desenvolvido consiste em uma aplicação em terminal, que ao iniciar o programa carrega os registros do arquivo TXT, cujas funcionalidades disponíveis são:

- Buscar peças por código;
- Inserir nova peça;
- Remover peça;
- Busca por prefixo;
- Verificar código com Bloom Filter;
- Filtrar peças por categorias;
- Mostrar estatísticas;
- Lista de peças em estoque crítico;
- Executar benchmark de busca por código;
- Executar benchmark de busca por prefixo;
- Executar benchmark entre Bloom Filter e Hash;

- Mostrar informações da Hash e da Hash Otimizada;
- Executar testes com restrições;
- Mostrar memória aproximada das estruturas.

3.2 Preparação do dataset

O dataset utilizado foi organizado por um arquivo TXT, com os campos e categorias separados e organizados. A base de registro foi estruturada de maneira suficiente para permitir uma comparação de desempenho eficiente mediante as estruturas e estatísticas utilizadas. Foi organizado pelo código da peça, descrição original, nome da peça, categoria, marca, modelo de moto, preço, quantidade em estoque e quantidade mínima. Essa organização foi mantida para a representação de que cada linha do arquivo seja uma peça individual dos dados, o código da peça é a chave principal que mantém a identificação de cada item dentro do campo de busca.

Durante a preparação base, foram adicionados campos sintéticos de dados, o que permite uma simulação de um sistema de controle de estoque, relacionados à quantidade em estoque e à quantidade mínima de cada peça. Dessa forma, o sistema executa operações para listagem de estoque crítico e análise de disponibilidade sobre o “status” da peça, o qual é calculado a partir da quantidade em estoque, o que possibilita uma classificação “sem estoque” sendo igual a zero, classificação de “estoque baixo” diante de uma quantidade menor ou igual a quantidade mínima e “disponível” pela quantidade referência mínima ou acima.

Após a preparação, os dados foram utilizados como entrada ao programa, fazendo uma leitura do TXT, organizando os campos de cada linha e convertendo os valores numéricos, assim armazenando esses registros em um vetor principal. A partir desse vetor, serão inicializadas as estruturas de dados.

3.3 Tratamento dos dados

No decorrer da leitura dos arquivos, o sistema realiza um tratamento dos dados antes de armazená-los em um vetor principal. As estruturas de dados implementadas não armazenam todos os campos da peça, mas apenas a chave e o índice correspondente ao registro no vetor. As implementações são:

- Remoção de espaços extras.
- Separação dos campos por pontos e vírgula.
- Conversão de valores inteiros.
- Ignora linhas vazias.

- Ignora registros de peças sem código.
- Recalcula o status de estoque com base disponibilidade de peças.

3.4 Operação do sistema

Analizando além do menu principal, o qual permite ao usuário executar as operações implementadas, são mantidas as operações básicas que fortificam a inserção, remoção e busca de peças, realizadas pelo código, mantido pela Lista Encadeada, tabela Hash, Hash otimizada ou Árvore AVL, permitindo comparar o comportamento das estruturas.

A inserção de novas peças é direcionada para o vetor e atualiza as estruturas como índice, já a remoção marca o registro como inativo no vetor e remove a chave das estruturas principais pelo código de peça. Já as estruturas auxiliares, Trie e o Bloom Filter, são reconstruídas com base nos registros ainda ativos, considerando que o Bloom Filter não realiza a remoção diretamente.

O sistema executa operações adicionais que reforçam a manipulação de análise de dados, sobre filtragem por categoria, a listagem de estoque, a busca por prefixo no nome da peça, verificação probabilística e a geração estatística, com média de preços, desvio padrão e análise de disponibilidade.

4 Desenvolvimento do Sistema

O sistema desenvolvido no trabalho foi implementado em linguagem C++, o que permite uma maior organização do código e facilita a manipulação de dados textuais, arquivos e estruturas dinâmicas. Como o dataset contém compôs de código da peça, descrição, nome, categorias, fabricante e modelo da moto, o uso do tipo “string” facilita no armazenamento e tratamento, além do uso de “vector” para os registros completos das peças, permitindo um crescimento dinâmico conforme os registros, sem alocação manual.

Outro exemplo relevante é sobre os meios “ifstream” e “ofstream” recursos para ler em formato TXT e garantir seus resultados, além dos recursos da biblioteca “chrono” para medição estatística de tempo dentro da busca, inserção e remoção, o que complementou para os dados exigidos pelos benchmarks.

4.1 Atuação dos registros

Cada peça do catálogo é representada por uma estrutura chamada “Peca”, tal estrutura armazena os principais atributos que contém no arquivo TXT em um campo booleano chamado “ativo”. Esse campo é utilizado para controlar em qual situação do registro esta o vetor principal, assim, quando uma peça é removida, ela passa a ser considerada inativa, deixando de participar nas análises de busca. O que preserva a posição

dos elementos do vetor, assim evita que a remoção de outros itens altere os índices dos registros já referenciados.

Após o carregamento do vetor principal, o código percorre os registros gerais ativos e insere as chaves nas estruturas de dados. Sendo assim, para a Lista Encadeada, tabela Hash, Hash otimizada e Árvore AVL, a chave é o código da peça, além de armazenar o índice correspondente ao registro no vetor. Diferente do Trie, recebendo o nome da peça, ideal ao prefixo, e o Bloom Filter pelo teste de provável existência, o que separa as estruturas de acordo com as finalidades.

4.2 Inserção de dados

Pela dinâmica da inserção de dados, pela forma manual, permite o cadastro de novas peças mesmo durante a execução do sistema, no qual o usuário informa um código novo, a descrição, nome da peça, categoria, fabricante, modelo, preço, quantidade em estoque e quantidade mínima.

Antes da inserção, é verificada a possível existência da peça adicionada, podendo ser cancelada, ou mesmo adicionada ao vetor e atualizando as estruturas. Além da mediação de tempo conforme a inserção de escolha, o que se realça é comparar o custo de atualização de todas as estruturas de dados utilizadas.

4.3 Busca de dados

A busca é realizada pela escolha da estrutura, o que mantém a diferença de aplicação e fortifica a comparação, podendo ser:

- Lista Encadeada.
- Tabela Hash.
- Hash Otimizada.
- Árvore AVL.

Diante da busca, o sistema informa se a peça foi encontrada e exibe os dados correspondentes, além da exibição do nome da estrutura escolhida, os dados estatísticos de complexidade, tempo de consulta e tratamento de repetição.

4.4 Remoção de dados

Pela análise do código geral da peça, a remoção é mantida pelo registro marcado como inativo no vetor e removido das estruturas principais. Nas auxiliares, é reestruturada pela dinâmica de ser reconstruída com bases nos registros ainda ativos, pois a atualização

é diferente, o que fortifica principalmente a estrutura Bloom Filter que, de maneira geral, não realiza a remoção direta, assim deve ser “lido” novamente.

4.5 Visualização dos dados

A visualização ocorre por um terminal, o sistema apresenta os dados completos da peça escolhida, informações resumidas por listagem, a estatística das peças, dados da Hash, medição do tempo de execução, comparações e resultados dos benchmarks e teste restritivo.

5 Benchmarks e Resultados

Os resultados foram obtidos por meio da implantação do sistema sobre o menu, onde os resultados apresentados foram obtidos por uma análise de um conjunto de 18.421 registros, mantendo a análise principalmente por opções de benchmarks, testes restritivos, informações da Hash e estimativa aproximada da memória. Onde muitas implementações mantiveram a ideia de avaliar o desempenho das estruturas em diferentes tarefas.

5.1 Busca por código

O principal Benchmark manteve em comparar a Lista Encadeada, tabela Hash, Hash Otimizada e Árvore AVL, tal comparação avaliou o desempenho das estruturas, onde resolvem o mesmo problema de localizar um registro por meio de um código, para análise o código 420240 foi mantido como padrão estatístico.

Tabela 1: Benchmark de busca por código

Estrutura	Tempo médio (s)	Complexidade
Lista Encadeada	0.0000517	$O(n)$
Tabela Hash	0.000000137	$O(1)$
Hash Otimizada	0.000000084	$O(1)$
Árvore AVL	0.00000042	$O(\log n)$

Diante dos resultados, é possível destacar que a Lista Encadeada tende a ter maior tempo de busca, o que é explicado pela forma sequencial de busca, percorrendo os elementos até encontrar a chave desejada. Destaca a complexidade $O(n)$, proporcional ao tempo de busca conforme o registro de dados.

A tabela Hash e a Hash otimizada tendem a ter tempos menores, o que mantém a função hash para calcular e encontrar a posição da chave e sendo eficaz na busca, mesmo que apresente colisões, as condições médias são apresentadas por $O(1)$. Já a Árvore AVL

apresenta um comportamento intermediário, com complexidade de $O(\log n)$, no qual se baseia na ideia de divisão e conquista.

5.2 Tabela Hash e Hash Otimizada

Tal comparação foi realizada para entender e comparar as funcionalidades e desempenhos de ambas. A tabela Hash “simples” possui um tamanho fixo, diferente da Hash otimizada que controla o fator de carga e redimensiona o tamanho de maneira automática ao ultrapassar 75% da ocupação.

Tabela 2: Comparação Tabela Hash e Hash Otimizada

Métrica	Tabela Hash	Hash Otimizada
Tamanho da tabela	1009	32319
Quantidade de elementos	18421	18421
Fator de carga	18.25	0.57
Colisões	17412	4187
Rehashes	Não se aplica	5

Como a Hash simples possui um tamanho fixo, a adaptação não é realizada, assim, quando inúmeros elementos são trabalhados ou quando eles aumentam, a tendência é que os dados sejam direcionados para posições já ocupadas, o que eleva o nível de colisões.

A Hash otimizada monitora o fator de carga, sendo essa a ideia de o “quão cheia” a tabela está, o que mantém um equilíbrio do tempo de busca e a memória. Assim, o “Rehash” apresenta a operação de redimensionamento, o que ajuda a manter o desempenho de busca $O(1)$, contudo, esse redimensionamento realça uma criação de a novo array, o que possui um custo adicional de memória.

5.3 Benchmark Lista e Trie

A Trie foi utilizada como uma estrutura auxiliar, dentro da busca por prefixo no nome das peças, e o desempenho foi em comparado com uma busca sequencial, por uma lista de dados ativos. Essa comparação é realizada quando ambas as estruturas contam os registros pelo nome das peças, definido 5 prefixos para fazer a busca (FILTRO, ÓLEO, BATERIA, PASTILHA e CABO).

Tabela 3: Benchmark de busca por prefixo

Estrutura	Tempo de Busca (s)
Lista	0.03279
Trie	0.000000001

Pela busca da lista, será preciso percorrer todos os registros e verificar os nomes que começam com o prefixo digitado. Com isso, o custo tende a crescer pela ideia e quantidade de peças cadastradas. Já a Trie percorre os caracteres do prefixo, com uma consulta mais adequada. Mas a ideia é o complemento da estrutura auxiliar que colabora para a funcionalidade do sistema.

5.4 Benchmark Bloom Filter e Tabela Hash

A estrutura auxiliar Bloom Filter caracteriza uma probabilística que realça se o código provavelmente existe nos âmbitos dos dados. Onde ela apresenta um falso positivo, a confirmação dos dados é feita pela Tabela Hash.

Tabela 4: Benchmark Bloom Filter x Tabela Hash

Estrutura	Tempo Total (s)	Resultado Analisado
Bloom Filter	0.000000001	5000
Tabela Hash	0.00407	5000
Falsos Positivos simulados	[Não se aplica]	433

Analisando o Bloom Filter, ele permite verificar se o código pode existir no conjunto de dados, contudo, a informação verdadeira é pela Hash, assim, o Bloom Filter mantém como estrutura auxiliar de baixo consumo de memória, e deve ter um complemento para confirmação do registro, interessante para embasamento de testes.

5.5 Resultados de inserção e remoção

Diante das características estatísticas, o sistema analisa o tempo de inserção e remoção das estruturas, quando uma peça é adicionada e depois inserida nas estruturas e quando é removida, marcando a peça como inativa.

Tabela 5: Tempo de inserção por estrutura

Estrutura	Tempo de Inserção (s)
Lista Encadeada	0.000000001
Tabela Hash	0.000000001
Hash Otimizada	0.000000001
Árvore AVL	0.000000001
Trie	0.0000000001

Tabela 6: Tempo de remoção por estrutura

Estrutura	Tempo de Remoção (s)
Lista Encadeada	0.000000003
Tabela Hash	0.0000000001
Hash Otimizada	0.0000000001
Árvore AVL	0.0000000001
Trie	0.107339
Bloom Filter	0.107339

Os resultados demonstram que cada estrutura possui um custo, a lista possui uma isenção simples, mas a remoção e busca exigem um percurso sequência mais longo e custoso. A Hash tem uma operação por chave, o que tende a ser eficaz, já a AVL tem um desempenho divisível e logarítmico. A reconstrução da Trie e do Bloom Filter possui um custo adicional, mas foi realizada dessa forma para manter uma consistência nas estruturas.

5.6 Memória Aproximada

O programa apresenta uma estimativa aproximada da memória (bytes) utilizada nas estruturas. No qual é considerado pela estimativa o tamanho estrutural de nós, ponteiros e vetores.

Tabela 7: Memória aproximada das estruturas

Estrutura	Memória Aproximada (bytes)
Lista Encadeada	884224
Tabela Hash	892280
Hash Otimizada	1142760
Árvore AVL	1031576
Trie	9582440
Bloom Filter	12501

O dado apresentado realça uma estimativa, comparando tendências gerais de consumo entre as estruturas. O Bloom Filter tende a apresentar um consumo baixo, pois é utilizado vetor de bits, já as estruturas baseadas em nós podem consumir mais memória pelo uso de ponteiros e para alocações dinâmicas.

6 Testes com Restrições

Testes com condições restritivas tiveram o objetivo de avaliar diferentes comportamentos das estruturas de dados, sobre situações menos favoráveis, aproximando de análises reais, podendo existir limitações de memória, maior carga de processamentos, atrasos na execução, restrição de dados ou algorítmica.

6.1 Restrição de memória

A restrição de memória foi mantida utilizando uma Tabela Hash como simulação, aplicando um tamanho reduzido, o que provoca maior concentração de elementos lidos em poucas posições, o que corresponde a categoria 1 relacionada ao item R2, tratando da restrição no tamanho máximo das estruturas de dados.

Assim, o número de colisões tende a aumentar. Esse teste aborda como uma estrutura Hash pode perder desempenho quando o espaço é insuficiente para distribuir os elementos.

Tabela 8: Restrição de Memória

Métricas	Resultados
Tamanho da Hash	101
Quantidade de colisões	18320

Ao reduzir o tamanho, os elementos passam a ser direcionados para as mesmas posições, assim, o desempenho pode ser afetado pelas altas colisões e limitando a memória de uso, prejudicando a busca de ser tão direta e passando a depender mais do percurso dentro dos encadeamentos.

6.2 Restrição de processamento

A restrição de processamentos é analisada por meio de uma grande quantidade de busca, utilizando a Tabela Hash, simulando elevadas cargas, correspondendo à categoria 2, relacionado ao item R9, tratando do aumento de cargas com número excessivo de operações, o que permite observar um comportamento de repetição da estrutura.

Tabela 9: Teste de restrição de processos

Métricas	Resultados
Quantidade de busca	100000
Tempo de busca	0.030525 (s)

O teste demonstra que a estrutura mantém um bom desempenho, mesmo que inúmeras operações estão sendo realizadas. Como a tabela Hash apresenta uma complexidade média de $O(1)$, é esperado um bom comportamento.

6.3 Restrição de latência

A latência realça uma simulação no programa por um atraso artificial durante a execução de buscas, permitindo uma demora adicional no processamento, podendo ser limitação do ambiente e concorrência de recursos, o que corresponde a categoria 3, diante do R13, forçando atrasos antes da execução.

Tabela 10: Teste de latência

Métricas	Resultados
Quantidade de busca	1000
Estrutura utilizada	Tabela Hash.
Tempo de atraso	0.007033 (s)

A ideia é que, mesmo com uma boa conduta da estrutura pela busca, o tempo final pode ser influenciado por fatores externos.

6.4 Restrição de dados

A restrição de dados é relacionada a presença de registros incompletos ou anômalos no dataset, correspondente à categoria 4, relacionada ao item R18 que trata da presença de valores anômalos, que dentro do dataset realça a presença de preços registrados por zero e modelos de motos não identificadas. Além do R20, que trata do uso de dados sintéticos, diante do sistema, alguns campos foram abordados para controle de estoque, como a quantidade em estoque e quantidade mínima, permitindo a simulação de disponibilidade de peças.

Tabela 11: Restrição de dados

Métricas	Resultados
Peças com preço zero	5052
Peças não identificadas	5684

A ideia da análise e teste desses dados é evitar a distorção de resultados, onde o sistema diferencia o preço médio bruto do preço válido, diferenciando os valores zerados.

6.5 Restrição algorítmica e estrutural

Restrição algorítmica e estrutural foi mantida por comparar diferentes estruturas que realizam a mesma tarefa, no caso a busca por código, correspondente a categoria 5, relacionada ao item R21, tratando a comparação entre estruturas eficientes e estruturas menos eficientes.

Comparando Lista Encadeada, Tabela Hash, Hash otimizada e Árvore AVL, que objetificam a influência da escolha de uma estrutura para o desempenho do sistema, que organiza e localiza de formas diferentes os dados.

Tabela 12: Teste de restrição algorítmica e estrutural

Métricas	Tempo (s)
Lista Encadeada	0.91832
Tabela Hash	0.001569
Hash otimizada	0.00000001
Árvore AVL	0.00000001

Esse desempenho foi mantido pela comparação de tempo de execução, analisando uma quantidade de 5000 buscas diante das estruturas. O teste evidencia que a escolha de uma estrutura deve estar relacionada ao tipo de operação, quanto por códigos ou busca ordenada, para ajudar no desempenho do sistema.

6.6 Escalabilidade

Como um teste complementar, a escalabilidade indica que a quantidade de buscas foi aumentada gradualmente, o que permite observar o tempo de execução da estrutura aumentando conforme a carga de operação.

Tabela 13: Teste de escalabilidade

Consultas	Lista	Hash	Hash Otimizada	Árvore AVL
1000	0.214761	0.00000001	0.00000001	0.00000001
3000	0.59091	0.00000001	0.0086240	0.0005060
5000	0.924205	0.00000001	0.00000001	0.0061120
10000	1.530102	0.0039520	0.0041160	0.0038770

7 Conclusão

O projeto desenvolvido permitiu aplicar diferentes estruturas de dados, com o objetivo de gerenciar e analisar um catálogo de peças de moto, utilizando um dataset

estruturado, que possibilitou realizar operações de cadastro, busca, remoção, filtragem, controle de estoque e análise estática. As estruturas utilizadas foram Lista Encadeada, Tabela Hash, Hash Otimizada e Árvore AVL, que reforçaram a busca por código.

A Trie e o Bloom Filter foram aplicações auxiliares, onde mantiveram a ideia de busca por prefixo e atividade probabilística. Além da implementação de uma estrutura melhorada, uma Hash otimizada, por redimensionamento automático a fim de reduzir colisões e controle de carga. Os testes restritivos e benchmarks também evidenciaram uma otimização e análise de comportamentos, dentre resultados e condições.

Diante das inúmeras diferenças de estruturas e observações, a Lista Encadeada apresenta comportamento menos eficiente para busca por código, diferente da busca por chave, como a Tabela Hash ou mesmo por busca balanceada. Além da utilidade de buscas textuais e da existência probabilística, que caracterizaram como boas propostas auxiliares. Testes de condições representaram uma análise adequada para o entendimento dos sistemas, revigorando diferentes comportamentos e exaltando a utilidade e condições das estruturas de dados.

Como parâmetros, inúmeras ações de mediação e remediação foram apresentadas, desde a influência direta do uso de memória sobre consumo, além de dados sintéticos que modificaram o âmbito do código e agregaram em uma definição ideal do programa, o que manteve especialmente os campos relacionados à quantidade em estoque e quantidade mínima. A proposta permitiu a interação do programa, desde a operação de dados e leituras do dataset, a geração de meios estatísticos, comparativos aos benchmarks e comportamentos das estruturas em diferentes situações.

Portanto, o desenvolvimento do projeto permitiu a compreensão, de forma prática, das diferentes ações das estruturas de dados, que influenciam diretamente no desempenho e organização do sistema. O trabalho demonstrou que a melhor escolha está ligada ao tipo de operação, além da quantidade de dados e ações restritivas para ambientes propícios. Assim, o projeto consolidou uma ação de aprendizagem das estruturas de dados, diante de suas aplicações e análises de informações.

Referências

OPENAI. *ChatGPT: conversa de apoio no desenvolvimento do código*. 2026. Disponível em: <https://chatgpt.com/share/6a3f62f1-20d4-83e9-b2bb-337125617f3e>. Acesso em: 20 jun. 2026.

LUSQUINO FILHO, Leopoldo André Dutra. **Estrutura de Dados: Conjuntos Dinâmicos e Listas Ligadas**. Material de aula da disciplina Estrutura de Dados. 2026.

LUSQUINO FILHO, Leopoldo André Dutra. **Estrutura de Dados: Árvores Binárias, Árvores de Busca e Árvores AVL**. Material de aula da disciplina Estrutura de Dados. 2026.

LUSQUINO FILHO, Leopoldo André Dutra. **Estrutura de Dados: Hashing e Tratamento de Colisões**. Material de aula da disciplina Estrutura de Dados. 2026.

LUSQUINO FILHO, Leopoldo André Dutra. **Proposta do Trabalho Final da Disciplina de Estrutura de Dados**. Documento de orientação do trabalho acadêmico. 2026.